

Web Development Fundamentals

Assignment: Frontend, Backend, and Web Architecture Concepts

Prepared by: Dhruv

Course: B.Tech (Computer Science) — Web Development / Full Stack Basics

Date: August 2026

Q1. Difference between Frontend, Backend, and Full-Stack Development

Web development is broadly divided into three areas based on which part of an application a developer works on: the part users see and interact with, the part that runs behind the scenes, and a combination of both.

Frontend Development

Frontend (client-side) development deals with everything the user sees and interacts with directly in the browser — layout, design, colours, buttons, forms, and animations. It is built using HTML, CSS, and JavaScript, often with frameworks such as React, Angular, or Vue.

- **Real-world example:** The visual design, menu, and 'Add to Cart' button on Amazon's website.

Backend Development

Backend (server-side) development handles the logic, database operations, authentication, and server configuration that power an application but are invisible to the user. Common backend technologies include Node.js, Python (Django/Flask), Java (Spring), and PHP, along with databases like MySQL or MongoDB.

- **Real-world example:** When you log in to Amazon, the backend verifies your password against the database and manages your session.

Full-Stack Development

A full-stack developer works on both frontend and backend, capable of building a complete application end-to-end — from the user interface to the server logic and database. The MERN stack (MongoDB, Express, React, Node.js) is a popular full-stack combination.

- **Real-world example:** A single developer building an entire food-ordering platform — the ordering screens (frontend), the order-processing and payment logic (backend), and the database that stores orders — is doing full-stack development.

Aspect	Frontend	Backend	Full-Stack
Focus	User interface & experience	Server logic & data	Both UI and server logic
Languages/Tools	HTML, CSS, JavaScript, React	Node.js, Python, Java, SQL	Combination of both
Visibility to user	Directly visible	Not visible	Both
Example Role	UI Developer	API/Server Developer	Full-Stack Engineer

Q2. Diagram: Client-Server Model in Web Architecture

The client-server model is the foundation of how the web works. The 'client' (a browser or app) sends a request to a 'server', which processes it and sends back a response. This exchange happens over a network using the HTTP/HTTPS protocol.

Client-Server Model in Web Architecture



The client sends a request over the Internet using HTTP/HTTPS; the server processes it and sends back a response.

Figure 1: Client-Server communication model showing request and response flow.

In simple terms: the client asks (requests), and the server answers (responds). This request-response cycle repeats every time a user visits a webpage, submits a form, or clicks a link.

Q3. How a Browser Requests and Displays a Web Page

When a user types a URL into the address bar and presses Enter, the browser performs a series of steps to fetch and render the page:

- **1. URL Parsing & DNS Lookup:** The browser breaks the URL into parts (protocol, domain name, path) and looks up the domain's IP address using DNS (Domain Name System).
- **2. Establishing Connection:** The browser opens a TCP connection to the server at that IP address, using HTTPS if secure, which also involves an SSL/TLS handshake.
- **3. Sending HTTP Request:** The browser sends an HTTP GET request asking for the specific page/resource.
- **4. Server Processing:** The web server processes the request (running backend code if needed, querying a database) and sends back an HTTP response, usually containing an HTML file along with a status code (e.g., 200 OK).
- **5. Rendering the Page:** The browser reads the HTML and builds the DOM (Document Object Model). It fetches linked resources such as CSS, JavaScript, images, and fonts, applying them via the CSSOM and rendering engine.
- **6. Painting & Display:** JavaScript runs to add interactivity, and the final page is displayed on the user's screen.

This entire process, from typing the URL to seeing the fully loaded page, typically takes a fraction of a second to a few seconds depending on network speed and page complexity.

Q4. Tools Required to Set Up a Web Development Environment

Setting up a productive web development environment requires several categories of tools, each serving a specific purpose:

Tool	Purpose
Code Editor / IDE (e.g., VS Code)	Writing, editing, and debugging code with syntax highlighting, extensions, and integrated terminal.

Tool	Purpose
Web Browser (e.g., Chrome, Firefox)	Running and testing the webpage; browser DevTools help inspect HTML/CSS/JS and debug errors.
Version Control System (Git)	Tracking code changes, collaborating with teams, and maintaining history of the project.
Hosting Platform (GitHub/GitLab)	Storing source code remotely and enabling collaboration and deployment (e.g., GitHub Pages, Vercel).
Package Manager (npm/yarn)	Installing and managing third-party libraries and dependencies for JavaScript projects.
Runtime Environment (Node.js)	Allows JavaScript to run outside the browser, needed for backend development and build tools.
Web Server (Apache/Nginx/Node)	Hosting and serving the website to users over the internet.
Database Software (MySQL/MongoDB)	Storing, retrieving, and managing application data.
Browser Developer Tools	Inspecting elements, debugging JavaScript, monitoring network requests, and checking performance.
API Testing Tool (Postman)	Testing backend APIs by sending requests and checking responses independently of the frontend.

Q5. Web Servers: Definition and Examples

A web server is software (often running on dedicated hardware) that stores, processes, and delivers web pages and other content to clients (browsers) upon request, typically using the HTTP or HTTPS protocol. It listens for incoming requests, retrieves or generates the requested resource, and sends it back as a response.

Commonly Used Web Servers

- **Apache HTTP Server:** One of the most widely used open-source web servers, known for flexibility and support for many programming languages via modules.
- **Nginx:** Known for high performance and efficient handling of many simultaneous connections; also used as a reverse proxy and load balancer.
- **Microsoft IIS (Internet Information Services):** A web server built into the Windows Server operating system, commonly used for hosting ASP.NET applications.
- **Node.js (with Express):** Uses the built-in Node.js environment (with frameworks like Express.js) to run JavaScript-based web servers.
- **LiteSpeed:** A lightweight web server that comes bundled with Apache, commonly used in shared hosting environments.

Q6. Roles in a Project: Frontend Developer, Backend Developer, Database Administrator

Role	Key Responsibilities
Frontend Developer	Designs and builds the user interface using HTML, CSS, and JavaScript; ensures the site is responsive, accessible, and visually consistent; implements interactivity and connects the UI to backend APIs.
Backend Developer	Builds server-side logic, APIs, and authentication systems; handles data processing, business rules, and integration with the database; ensures security, scalability, and performance of the server.
Database Administrator (DBA)	Designs, implements, and maintains the database structure (schema); manages data backups, security, and access permissions; optimizes queries for performance and ensures data integrity.

Q7. Installing and Configuring VS Code for HTML, CSS, and JavaScript Development

Visual Studio Code (VS Code) is a free, lightweight, and highly extensible code editor widely used for web development. Below are the steps to install and configure it:

- **1. Download VS Code:** Go to the official website <https://code.visualstudio.com> and download the installer for your operating system (Windows/macOS/Linux).
- **2. Install:** Run the installer, accept the license agreement, and complete the setup wizard (on Windows, check 'Add to PATH' during installation).
- **3. Install Useful Extensions:** Open VS Code and go to the Extensions panel (Ctrl+Shift+X). Install: 'Live Server' (to preview HTML pages with auto-reload), 'Prettier' (code formatter), 'ESLint' (JavaScript linting), and 'HTML CSS Support'.
- **4. Set Up a Project Folder:** Create a project folder, open it in VS Code (File → Open Folder), and create index.html, style.css, and script.js files.
- **5. Configure Live Server:** Right-click index.html and select 'Open with Live Server' to preview the page in the browser with automatic refresh on save.

Note: As this is a text-based document, an actual screenshot of the VS Code setup could not be captured in this environment. In practice, after completing the steps above, you would take a screenshot of your VS Code window (showing the file explorer, an open HTML file, and the integrated terminal) using your operating system's screenshot tool (e.g., Windows: Win + Shift + S) and insert it here.

Q8. Static vs Dynamic Websites

Websites can be broadly classified into static and dynamic based on how their content is generated and delivered to the user.

Aspect	Static Website	Dynamic Website
Content	Fixed; same for every visitor	Changes based on user, time, or input
Technology	HTML, CSS, (minimal JS)	HTML/CSS/JS + server-side language & database
Data Source	Files stored directly on server	Generated from a database at request time
Update Process	Requires manually editing files	Content updates automatically via backend/admin panel
Interactivity	Limited	High — supports logins, forms, personalization
Example	A simple portfolio or brochure website	An e-commerce site like Amazon or a social media feed

Example of a static website: A personal resume/portfolio page that displays the same information to every visitor.

Example of a dynamic website: An online food ordering platform where the menu, prices, and order history differ for each logged-in user and are fetched live from a database.

Q9. Five Web Browsers and Their Rendering Engines

A rendering engine (or browser engine) is the software component that interprets HTML, CSS, and JavaScript to display a webpage visually. Different browsers use different rendering engines, which can cause minor differences in how the same page appears.

Browser	Rendering Engine	Notes
Google Chrome	Blink (fork of WebKit)	Fast, widely used; also powers many Chromium-based browsers.

Browser	Rendering Engine	Notes
Mozilla Firefox	Gecko	Open-source engine developed independently by Mozilla; strong standards compliance.
Apple Safari	WebKit	Default browser on macOS/iOS; optimized for Apple devices and energy efficiency.
Microsoft Edge	Blink (Chromium-based)	Switched from its own EdgeHTML engine to Chromium's Blink in 2020.
Opera	Blink (Chromium-based)	Also uses Blink since it moved to the Chromium base in 2013.

Because Blink and WebKit share a common ancestry, Chrome, Edge, and Opera tend to render pages very similarly, while Firefox (Gecko) and Safari (WebKit) sometimes render subtle CSS or layout details differently — which is why developers test websites across multiple browsers to ensure cross-browser compatibility.

Q10. Labeled Diagram: Client, Server, Database, and API Flow

The diagram below illustrates the basic architecture of a modern web application, showing how the client, server, API layer, and database interact to deliver a response to the user.

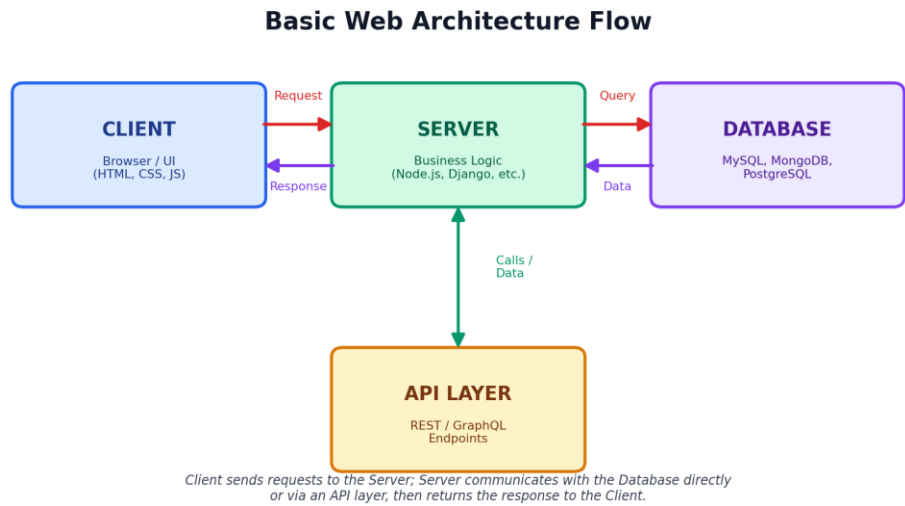


Figure 2: Basic web architecture flow — Client, Server, API, and Database.

Flow explanation:

- 1. The client (browser/app) sends an HTTP request to the server, e.g., asking to view a product page.
- 2. The server processes the request and, if needed, calls the API layer to fetch or update specific data.
- 3. The API layer communicates with the database using queries to retrieve or store the required data.
- 4. The database returns the requested data to the API/server, which formats it and sends the final response back to the client for display.